

Laboratory Work 4

Student: Orynassar Nurali

Task 1. Retrieve all airline names in uppercase.

I retrieved all airline names in uppercase using the UPPER() function.

Query

Query History

Scratch Pad X

1

SELECT *

2

FROM airline

3

LIMIT 5;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 5

Page No: 1 of

	airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	KC	Air Astana	KAZAKHSTAN	2026-09-22 06:51:39.460902	2026-09-22 06:51:39.460902
2	2	FS	FlyArystan	KAZAKHSTAN	2026-09-22 06:51:39.460902	2026-09-22 06:51:39.460902
3	3	TK	Turkish Airlines	TURKEY	2026-09-22 06:51:39.460902	2026-09-22 06:51:39.460902
4	4	EK	Emirates	UNITED ARAB EMIRAT...	2026-09-22 06:51:39.460902	2026-09-22 06:51:39.460902
5	6	KZ	KazAir	TURKEY	2026-09-28 20:44:48.567733	2026-09-28 20:45:38.773551

Task 2. Replace "Air" with "Aero".

I replaced every occurrence of the word "Air" in airline names with "Aero" using REPLACE().

Query

Query History

1

SELECT UPPER(airline_name) AS airline_name

2

FROM airline;

Data Output

Messages

Notifications

Showing rows: 1 to 10

	airline_name text
1	AIR ASTANA
2	FLYARYSTAN
3	TURKISH AIRLIN...
4	EMIRATES

Task 3. Find flight numbers that correspond with both airline 1 and airline 2.

I grouped flights by flight_id and used HAVING with COUNT(DISTINCT airline_id) to find flight numbers associated with both airlines.

Query Query History

```
1 SELECT flight_id
2 FROM flights
3 WHERE airline_id IN (1, 2)
4 GROUP BY flight_id
5 HAVING COUNT(DISTINCT airline_id) = 2;
```

Data Output Messages Notifications

flight_id
[PK] integer

Task 4. Retrieve airports containing "Reginal" and "Air".

I used ILIKE with two conditions to search for airport names containing both patterns. The current database returned no matching rows.

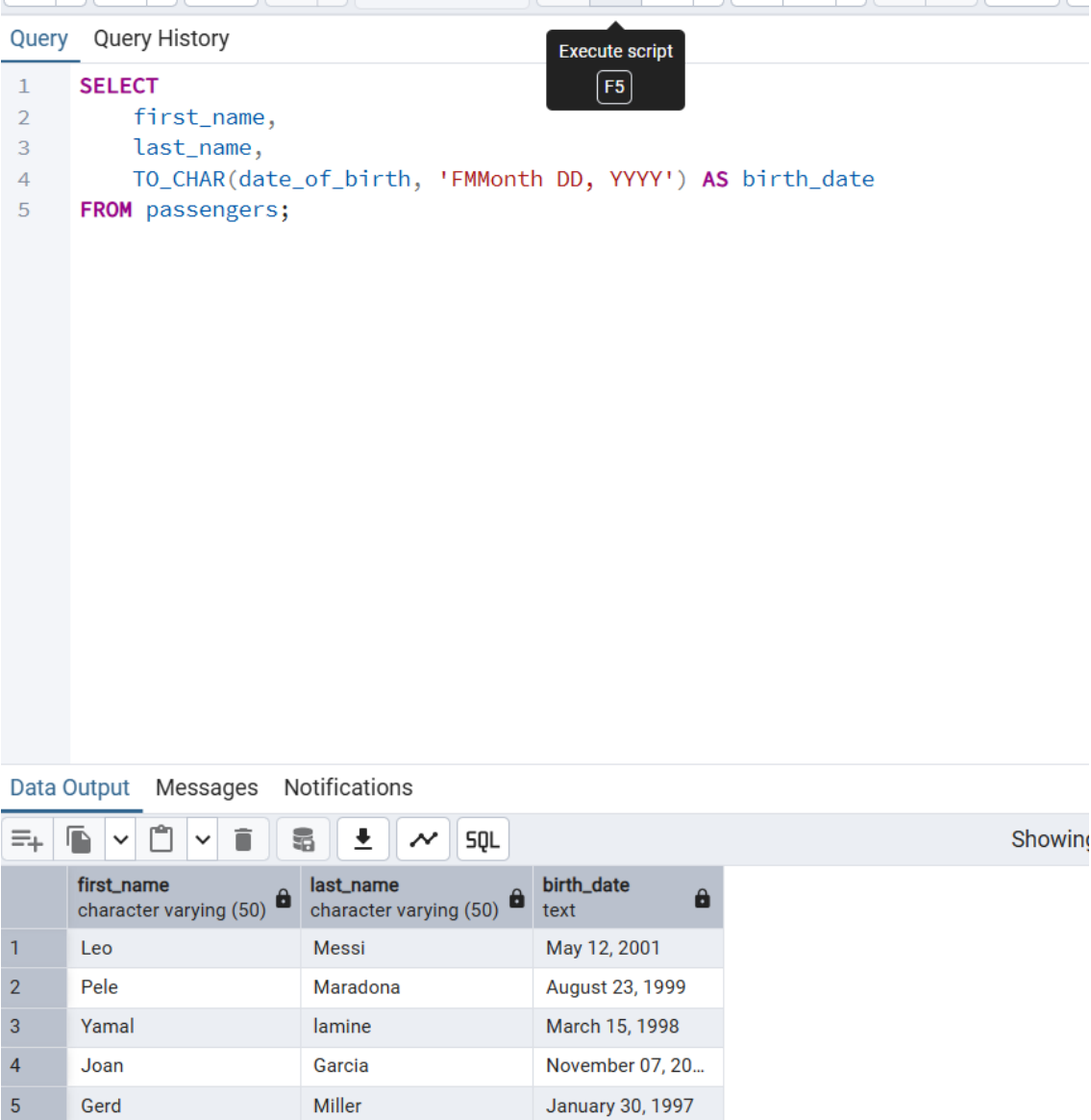
The screenshot shows a SQL query editor interface. At the top, there is a toolbar with various icons for editing and executing queries. Below the toolbar, the main area is divided into two tabs: "Query" and "Query History". The "Query" tab is active, displaying the following SQL query:

```
1 SELECT airport_name
2 FROM airport
3 WHERE airport_name ILIKE '%Reginal%'
4 AND airport_name ILIKE '%Air%';
```

Below the query editor, there is a section for "Data Output", "Messages", and "Notifications". The "Data Output" tab is active, showing a table with one column named "airport_name" and a data type of "character varying (50)". The table is currently empty, indicating that no rows were returned by the query.

Task 5. Retrieve passenger names and format birth dates.

I selected passenger first and last names and formatted date_of_birth as "Month DD, YYYY" using TO_CHAR().



The screenshot shows a SQL IDE interface. At the top, there are tabs for "Query" and "Query History". Below the tabs, a query is written in a text editor:

```
1 SELECT
2     first_name,
3     last_name,
4     TO_CHAR(date_of_birth, 'FMMonth DD, YYYY') AS birth_date
5 FROM passengers;
```

To the right of the query editor, there is a dark button labeled "Execute script" with a keyboard shortcut "F5".

Below the query editor, there are tabs for "Data Output", "Messages", and "Notifications". The "Data Output" tab is active, showing a table of results. The table has four columns: "first_name", "last_name", and "birth_date". The "first_name" and "last_name" columns are of type "character varying (50)" and the "birth_date" column is of type "text". The table contains five rows of data:

	first_name character varying (50)	last_name character varying (50)	birth_date text
1	Leo	Messi	May 12, 2001
2	Pele	Maradona	August 23, 1999
3	Yamal	Iamine	March 15, 1998
4	Joan	Garcia	November 07, 20...
5	Gerd	Miller	January 30, 1997

Task 6. Find flights that have been delayed.

I compared actual arrival time with scheduled arrival time. A flight is delayed when the actual arrival is later.

QueryQuery History

123

```
SELECT flight_id
FROM flights
WHERE act_arrival_time > sch_arrival_time;
```

Data OutputMessagesNotifications

SQL

Showin

	flight_id
	[PK] integer
1	1
2	2
3	3
4	4

Task 7. Find flights that arrived late and show the delay.

I calculated the delay by subtracting scheduled arrival time from actual arrival time.

Query

Query History

1

2

3

4

5

6

7

SELECT

flight_id,

sch_arrival_time,

act_arrival_time,

act_arrival_time - sch_arrival_time AS delay

FROM flights

WHERE act_arrival_time > sch_arrival_time;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing rows: 1 to 4

	flight_id [PK] integer	sch_arrival_time timestamp without time zone	act_arrival_time timestamp without time zone	delay interval
1	1	2026-09-25 10:00:00	2026-09-25 10:05:00	00:05:00
2	2	2026-09-26 13:00:00	2026-09-26 13:05:00	00:05:00
3	3	2026-09-27 15:00:00	2026-09-27 15:10:00	00:10:00
4	4	2026-09-28 18:00:00	2026-09-28 18:05:00	00:05:00

Task 8. Show airlines from France, Portugal or Poland created in the required period.

I filtered airline_country with IN and limited created_at to the required date range. The current database returned no matching rows.

Query

Query History

1

2

3

4

5

6

7

8

SELECT

airline_name,

airline_country,

created_at

FROM airline

WHERE airline_country IN ('FRANCE', 'PORTUGAL', 'POLAND')

AND created_at >= '2023-11-01'

AND created_at < '2024-04-01';

Data Output

Messages

Notifications

SQL

airline_name

airline_country

created_at

character varying (50)

character varying (50)

timestamp without time zone

Task 9. Find the top 3 overweighted baggage above 25 kg.

I filtered baggage heavier than 25 kg and sorted it from highest to lowest. The current database has no baggage above 25 kg.

Query

Query History

1

2

3

4

5

6

7

8

SELECT

baggage_id,

weight_in_kg,

booking_id

FROM baggage

WHERE weight_in_kg > 25

ORDER BY weight_in_kg DESC

LIMIT 3;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📶

SQL

baggage_id

[PK] integer

weight_in_kg

numeric (4,2)

booking_id

integer

Task 10. Find the youngest passenger's full name.

I sorted date_of_birth in descending order and selected the first row. The result is Leo Messi.

Query

Query History

Execute script
F5

1

2

3

4

5

6

SELECT

first_name,

last_name

FROM passengers

ORDER BY date_of_birth DESC

LIMIT 1;

Data Output

Messages

Notifications

Showing rows: 1

	first_name character varying (50)	last_name character varying (50)
1	Leo	Messi

Task 11. Find the cheapest booking price on each booking platform.

I grouped bookings by booking_platform and used MIN(ticket_price) to find the cheapest price for each platform.

Query

Query History

1

2

3

4

5

6

SELECT

booking_platform,

MIN(ticket_price) AS cheapest_price

FROM booking

GROUP BY booking_platform

ORDER BY booking_platform;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

St

	booking_platform character varying (50) 🔒	cheapest_price numeric 🔒
1	Mobile App	218.50
2	Travel Agency	598.00
3	Website	172.50

Task 12. Return airlines whose airline_code contains a digit.

I used the PostgreSQL regular expression ~ '[0-9]' to search for a digit in airline_code. The current result is empty.

Query Query History

```
1 SELECT
2     airline_name,
3     airline_code
4 FROM airline
5 WHERE airline_code ~ '[0-9]';
```

Data Output Messages Notifications

									SQL
airline_name		airline_code							
character varying (50)		character varying (30)							

Task 13. List the top 5 most recently created airlines.

I sorted airlines by created_at in descending order and limited the result to five rows.

NO LIMIT

Query

Query History

1

2

3

4

5

6

7

8

9

SELECT

airline_id,

airline_name,

airline_code,

airline_country,

created_at

FROM airline

ORDER BY created_at DESC

LIMIT 5;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 5

	airline_id [PK] integer	airline_name character varying (50)	airline_code character varying (30)	airline_country character varying (50)	created_at timestamp without time zone
1	7	AirEasy	AE	FRANCE	2026-09-28 20:46:19.547541
2	9	FlyFly	FF	POLAND	2026-09-28 20:46:19.547541
3	8	Dana	FH	BRAZIL	2026-09-28 20:46:19.547541
4	6	KazAir	KZ	TURKEY	2026-09-28 20:44:48.567733
5	5	Global Airways	LH	United Kingdom	2026-09-22 06:51:39.460902

Task 14. Return rows with booking_id between 200 and 300 and check_result not equal to Checked.

I used BETWEEN for the inclusive booking_id range and <> for the not-equal condition. The current result is empty.

```
SELECT *
FROM baggage_check
WHERE booking_id BETWEEN 200 AND 300
AND check_result <> 'Checked';
```

Input

Messages

Notifications

▼

📋

▼

🗑

📦

⬇

📈

SQL

baggage_check_id PK integer ✎	check_result character varying (50) ✎	created_at timestamp without time zone ✎	updated_at timestamp without time zone ✎	booking_id integer ✎	passenger_id integer ✎
--	--	---	---	---	---

Task 15. Find baggage checks updated earlier than creation in the same month.

I compared the months with DATE_TRUNC() and then checked that updated_at is earlier than created_at. The current result is empty.

Query

Query History

↗

Scratch Pad x

1

2

3

4

SELECT *

FROM baggage_check

WHERE DATE_TRUNC('month', updated_at) = DATE_TRUNC('month', created_at)

AND updated_at < created_at;

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑

📦

📥

📈

SQL

baggage_check_id	check_result	created_at	updated_at	booking_id	passenger_id
[PK] integer	character varying (50)	timestamp without time zone	timestamp without time zone	integer	integer